

LLVM Pass and Code Instrumentation

Prof. Moonzoo Kim,
School of Computing, KAIST

Motivating Example

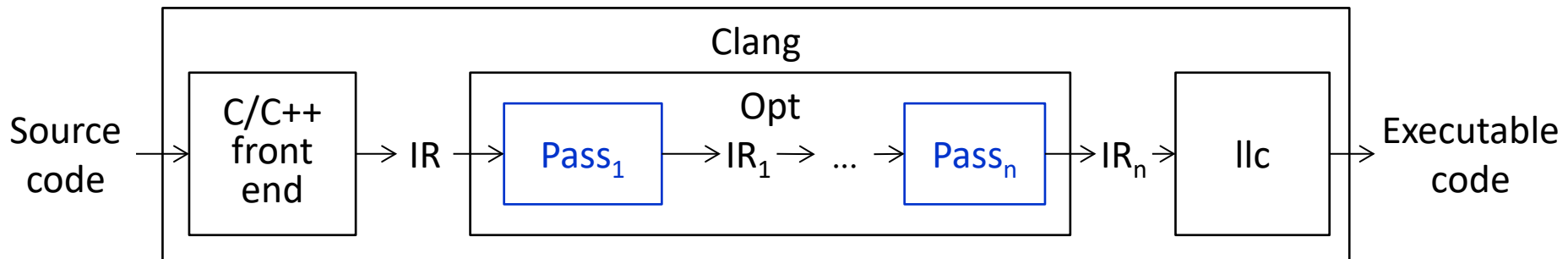
- (30 pts) Find bugs in the following program that has multiple bugs
 - Write down buggy lines, bugs, and explain the bugs as many as possible
 - Write down a code instrumentor using Clang which inserts `assert()` to report runtime failures due to the bugs you detected.
- For example, to report a div-by-zero crash, your code should insert `assert(z!=0)` immediately before `x=y/z`;

```
//example1.c
#include <malloc.h>
#include <stdio.h>
#include <string.h>
void f() {
    char* mem = NULL;
    int length;
    char buf[100];
    // file descriptor 0 is connected to keyboard
    read(0, &length, sizeof(int));
    int r=read(0, &buf,length>100 ?
        100:length);
    mem = malloc(r + 1);
    buf[r] = 0;
    strcpy(mem, buf);
    printf(mem);
    fflush(stdout);
}
```

Which tool do you prefer for the task? Clang? LLVM IR?

Pass in LLVM

- A Pass receives an LLVM IR and performs analyses and/or transformations.
 - Using `opt`, it is possible to run each Pass.
- A Pass can be executed in a middle of compiling process from source code to binary code.
 - The pipeline of Passes is arranged by Pass Manager

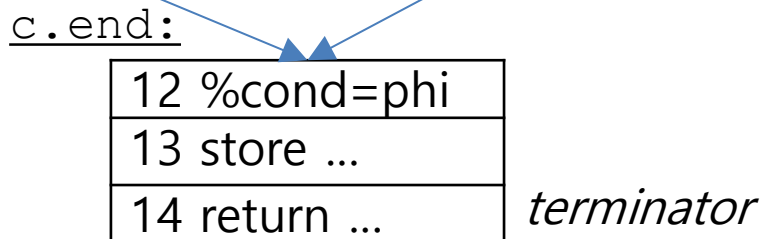
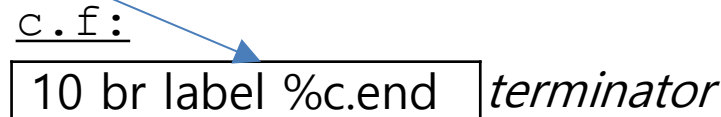
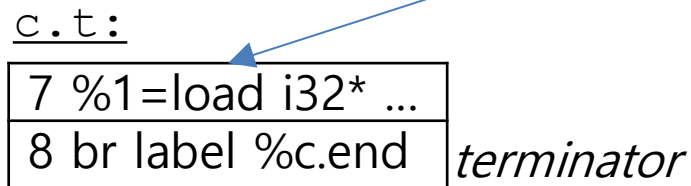
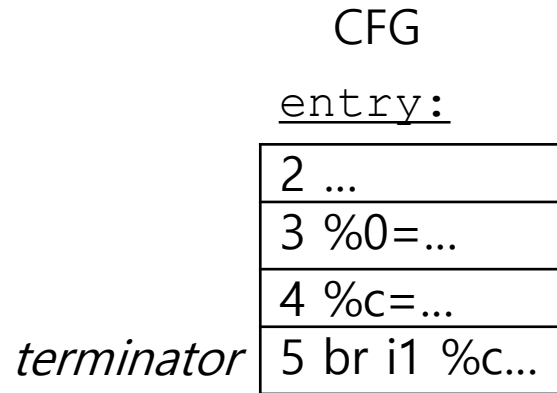


Control Flow Graph (CFG) at LLVM IR

```
int f() {  
    int y;  
    y = (x > 0) ? x : 0 ;  
    return y;  
}
```



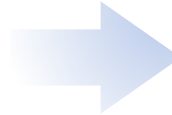
```
1 entry:  
2 ...  
3 %0 = load i32, ptr %x  
4 %c = icmp sgt i32 %0 0  
5 br i1 %c, label %c.t, %c.f  
  
6 c.t:  
7 %1 = load i32, ptr %x  
8 br label %c.end  
  
9 c.f:  
10 br label %c.end  
  
11 c.end:  
12 %cond = phi i32 [%1, %c.t], [0, %c.f]  
13 store i32 %cond, ptr %y  
14 return i32 %cond
```



Example Pass

- Let's create *IntWrite* that aim to monitor all history of 32-bit integer variable updates (definitions)
 - Produces a text file where it records which variable is defined as which value at which code location.
- IntWrite instruments a target program to insert a *probe* before every integer writing operation, which extracts runtime information

```
01 int f(int x) {  
...  
10     y = x ;  
11     z = y + x ;
```



```
void _probe_(int l,char *fn,int v){ ...  
    fprintf(fp,"Store %d in %s @line %d\n",  
            v,fn,l) ;}  
  
...  
_probe_(10, "f", x) ;  
10 y = x ;  
_probe_(11, "f", y+x) ;  
11 z = y + x ;
```

Module Class

- A `Module` instance stores all information related to the LLVM IR created by a target program file (functions, global variables, etc.)
- APIs (public methods)
 - `getModuleIdentifier()`: return the name of the module
 - `getFunction(StringRef Name)`: return the `Function` instance whose identifier is `Name` in the module
 - `getOrInsertFunction(StringRef Name, Type *ReturnType, ...)`: add a new `Function` instance whose identifier is `Name` to the module
 - `getGlobalVariable(StringRef Name)`: return the `GlobalVariable` instance whose identifier is `Name` in the module
 - `functions()`: return an iterator that iterates through functions in the module.

Type Class

- A `Type` instance is used for representing the data type of registers, variables, and function arguments.
- Static members
 - `Type::getVoidTy(...)`: void type
 - `Type::getInt8Ty(...)`: 8-bit unsigned integer (char) type
 - `Type::getInt32Ty(...)`: 32-bit unsigned integer type
 - `Type::getDoubleTy(...)`: 64-bit IEEE floating pointer type
 - `PointerType::get(...)`: opaque pointer type



PassInfoMixin Class (1/2)

- It is the class that you should inherit to make your own Pass.
- This example iterates through all basic blocks in all functions and executes `runOnBasicBlock` for each basic block.

```
1 class IntWrite: public PassInfoMixin<IntWrite> {
2 public:
3     PreservedAnalyses run(Module &M, ModuleAnalysisManager &MAM) {
4         for (llvm::Function &F : M) {
5             for (llvm::BasicBlock &BB : F) {
6                 runOnBasicBlock(BB) ;
7             }
8         }
9         return PreservedAnalyses::none();
10    }
    ...
```



Example

- IntWrite should insert a new function `_init_` at the beginning of the target program's main function
 - `_init_()` is to open an output file

```
01 bool doInitialization(Module &M) {
02     if (M.getFunction(StringRef("_init_")) != NULL) {
03         errs() << "Error: function _init_() already exists.\n" ;
04         exit(1) ;
05     }                                     check if _init_() already exists

07     FunctionType * fty = FunctionType::get(Type::getVoidTy(ctx), false) ;
08     p_init = M.getOrInsertFunction("_init_", fty) ;
09     ...
10     return true ;                         add a new declaration _init_()
11 } // doInitialization.
```

Instruction Class

- An `Instruction` instance contains the information of an LLVM IR instruction.
- Each type of instruction has a subclass of `Instruction` (e.g. `LoadInst`, `BranchInst`)
- APIs
 - `getOpcode()`: returns the opcode which indicates the instruction type
 - `getOperand(unsigned i)`: return the i-th operand
 - `getDebugLoc()`: obtain the debugging data that contains the information on the corresponding code location
 - `isTerminator()`, `isBinaryOp()`, `isCast()`, ...



Example

```
01 bool runOnBasicBlock(BasicBlock &B) {
02     for(BasicBlock::iterator i = B.begin(); i != B.end(); i++){
03         if(i->getOpcode() == Instruction::Store &&
04             i->getOperand(0)->getType() == Type::getInt32Ty(ctx)){
05             StoreInst * st = dyn_cast<StoreInst>(i);
06             int loc = st->getDebugLoc().getLine(); //code location
07             Value * var = st->getPointerOperand(); //variable
08             Value * val = st->getOperand(0); // value
09             /* insert a function call */
10         }
11     }
12     return true ;
13 }
```

How to Insert New Instructions

- `IRBuilder` class provides a uniform API for inserting instructions to a basic block.
 - `IRBuilder(Instruction *p)`: create an `IRBuilder` instance that can insert instructions **right before** `Instruction *p`
- APIs
 - `CreateAdd(Value *LHS, Value *RHS, ...)`: create an add instruction whose operands are `LHS` and `RHS` at the predefined location, and then returns the `Value` instance of the target operand
 - `CreateCall(Value *Callee, Value *Arg, ...)`: add a new call instruction to function `Callee` with the argument as `Arg`
 - `CreateSub()`, `CreateMul()`, `CreateAnd()`, ...

Value Class

- A `Value` is a super class of all entities in LLVM IR such as a constant, a register, a variable, and a function.
- The register defined by an `Instruction` is represented as a `Value` instance.
- APIs
 - `getType()`: returns the `Type` instance of a `Value` instance.
 - `getName()`: return the name from the source code.

Example

```
IRBuilder<> * IRB;

00 if(i->getOpcode() == Instruction::Store &&
01     i->getOperand(0)->getType() == Type::getInt32Ty(ctx) {

02     StoreInst * st = dyn_cast<StoreInst>(i);

03     int loc = st->getDebugLoc().getLine(); //code location
04     Value * val = st->getOperand(0); // target register
05
06     IRB->SetInsertPoint(&(*i));
07     Value * args[3] ;
08     args[0] = ConstantInt::get(intTy, loc, false) ;
09     args[1] = IRB->CreateGlobalString(funcname) ;
10     args[2] = val ;
11     IRB->CreateCall(p_probe, args, Twine(""));
    // p_probe should be created before by using
    // getOrInsertFunction() and target code should be compiled
    // with the function definition pointed by p_probe.
    // See IntWrite.cpp and IntWrite.c which contains the
    // definition of probe function

12 }
```



intwrite-pass.cpp

LLVM Fuction pass to monitor writing integer values at runtime
- Pass is compiled as a shared library and stored in the LLVM library directory



intwrite-rt.c

Runtime module that will be used by the pass
- Compiled as an object file and stored in the LLVM library directory

```
// intwrite-rt.c runtime module

#include <stdio.h>
#include <stdlib.h>

FILE * fp ;

extern void _final_() { fclose(fp) ;}

extern void _init_() { fp = fopen("log", "w") ; atexit(_final_) ;}

extern void _probe_(int line, char *func, int val) {
    if(line!=-1) fprintf(fp, "Store value %d in Function %s, line %d\n",val,func,line) ;
    else fprintf(fp, "Store value %d in unknown location\n", val) ;
}
```



intwrite-pass.cpp

LLVM Fuction pass to monitor writing integer values at runtime
- Pass is compiled as a shared library and stored in the LLVM library directory

```
// intwrite-pass.cpp LLVM pass
```

```
class IntWrite: public PassInfoMixin<IntWrite> { ... }
```

```
/* The code in the remaining part is to register this Pass to  
 * LLVM Pass Manager such that LLVM/Clang can use it. */
```

```
extern "C" ::llvm::PassPluginLibraryInfo llvmGetPassPluginInfo() {  
    return {LLVM_PLUGIN_API_VERSION, // Plugin API version  
            "IntWritePassPlugin",    // Plugin name  
            LLVM_VERSION_STRING,     // LLVM version  
            [](PassBuilder &PB) {  
                // Register module-level pass  
                PB.registerPipelineStartEPCallback(  
                    [](ModulePassManager &MPM, OptimizationLevel _level) {  
                        MPM.addPass(IntWrite());  
                        return true;  
                    })  
            });  
};  
}
```

```

class IntWrite: public PassInfoMixin<IntWrite> {
public:
    PreservedAnalyses run(Module &M, ModuleAnalysisManager &MAM) {
        if (!doInitialization(M)) {
            errs() << "Error: initialization of intwrite pass failed.\n" ; exit(1) ;
        }
        /* Invoke runOnBasicBlock() for each basic block under F. */
        for (llvm::Function &F : M) {
            for (llvm::BasicBlock &BB : F) {
                runOnBasicBlock(BB) ;
            }
        }
        return PreservedAnalyses::none();
    }

    bool doInitialization(Module &M) { ... }
    bool runOnBasicBlock (BasicBlock &B) {
        /* This function is invoked for each basic block
           * in the function.*/

        ...
    }
    // These variables are to store the type instances for primitive types.
    Type *intTy, *ptrTy, *voidTy, *boolTy ;

    FunctionCallee p_init ; // points to the function instance of _init_.
    FunctionCallee p_probe ; // points to the function instance of _probe_.
    IRBuilder<> * IRB;
};

```

```

bool doInitialization(Module &M) {
    /* check if there is a function in a target program that conflicts
     * with the probe functions */
    if (M.getFunction(StringRef("_init_")) != NULL) {
        errs() << "Error: function _init_() already exists.\n" ; exit(1) ; }
    if (M.getFunction(StringRef("_probe_")) != NULL) {
        errs() << "Error: function() already exists.\n" ; exit(1) ; }

    /* store the type instances for primitive types */
    llvm::LLVMContext &ctx = M.getContext() ;
    intTy = Type::getInt32Ty(ctx) ;
    ptrTy = PointerType::get(ctx, 0);
    voidTy = Type::getVoidTy(ctx) ;
    boolTy = Type::getInt1Ty(ctx) ;

    /* add a new declaration of function _init_ which has no argument */
    FunctionType * fty = FunctionType::get(voidTy, false) ;
    p_init = M.getOrInsertFunction("_init_", fty) ;

    /* add a new declaration of function _probe_ which accept three
     * arguments (i.e., int, char *, and int) */
    Type * args_types[3] ;
    args_types[0] = intTy ; //Type::getInt32Ty(*ctx) ;
    args_types[1] = ptrTy ; //Type::getInt8PtrTy(*ctx) ;
    args_types[2] = intTy ; //Type::getInt32Ty(*ctx) ;
    p_probe = M.getOrInsertFunction("_probe_",
        FunctionType::get(voidTy, ArrayRef<Type *>(args_types), false)) ;

    /* add a function call to _init_ at the beginning of
     * the main function*/
    Function * mainFunc = M.getFunction(StringRef("main"));
    if (mainFunc != NULL) { return true ; }
    IRB = new IRBuilder<>(ctx);
    IRB->SetInsertPoint(mainFunc->getEntryBlock().getFirstNonPHIOrDbgOrLifetime());
    IRB->CreateCall(p_init, {}) ;
    return true ;
} // doInitialization.

```

```

bool runOnBasicBlock (BasicBlock &B) {
    /* This function is invoked by runOnFunction() for each basic block
       * in the function. Note that this is not invoked by LLVM and different
       * from runOnBasicBlock() of BasicBlockPass.*/

    StringRef funcname = "unknown";
    DISubprogram * disubp = B.getParent()->getSubprogram();
    if (disubp) { funcname = disubp->getName(); }

    for (BasicBlock::iterator i = B.begin() ; i != B.end() ; i++) {
        /* for each instruction of the basic block in order */

        if (i->getOpcode() == Instruction::Store) {
            if (i->getOperand(0)->getType() == intTy) {
                StoreInst * st = dyn_cast<StoreInst>(i) ;

                /* add a function call to _probe_ right before
                   * a store instruction on an integer variable. */

                const DebugLoc &debugloc = st->getDebugLoc();

                int loc = -1;
                if (debugloc) { loc = debugloc.getLine(); }

                Value * val = st->getOperand(0) ; // the value to be assigned.

                IRB->SetInsertPoint(&(*i));

                Value * args[3] ;
                args[0] = ConstantInt::get(intTy, loc, false) ; // location of store instruction.
                // create a new string constant of the function name, and get the pointer to it.
                args[1] = IRB->CreateGlobalString(funcname) ;
                args[2] = val ; // the value to be assigned to the variable.
                IRB->CreateCall(p_probe, args, Twine("")) ;
                continue ;
            }
        }
    }
    return true ;}

```

More Information

- Writing an LLVM Pass
 - <https://llvm.org/docs/WritingAnLLVMNewPMPass.html>
- LLVM API Documentation
 - <http://llvm.org/doxygen/>